



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

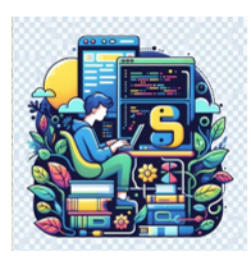
Computer Applications

PROGRAMMING WITH PYTHON

Data Ingestion and Multiprocessing Web Scrapping Sensor Data as Source

Lekha A

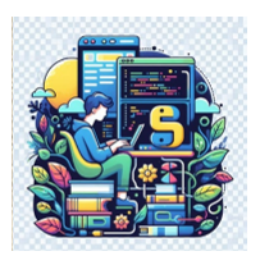
Computer Applications



PROGRAMMING WITH PYTHON

Web Scraping

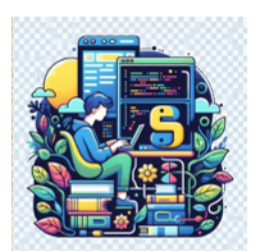
- Web scraping refers to the method of retrieving data from websites, usually through the use of automated tools.



PROGRAMMING WITH PYTHON

Web Scraping – Static Pages

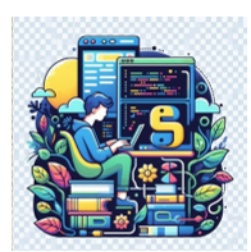
- In the case of static sites, the HTML documents sent by the server generally encompass all (or most) of the information needed.
- These pages might still employ JavaScript for certain minor client-side interactions.
- However, the content obtained from the server is essentially the full page that appears in the browser.



PROGRAMMING WITH PYTHON

Web Scraping – Dynamic Pages

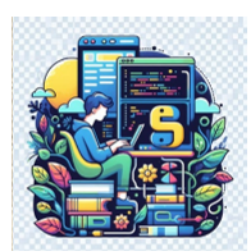
- Dynamic sites are heavily reliant on JavaScript to load and/or render data in the browser.
- The initial HTML document returned by the server often contains minimal actual data.



PROGRAMMING WITH PYTHON

Web Scraping – Dynamic Pages

- Instead, data is retrieved and rendered by JavaScript either upon the first page load or after user interactions
 - Infinite scrolling
 - or
 - dynamic pagination



PROGRAMMING WITH PYTHON

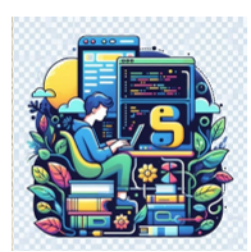
Web Scraping – Dynamic Pages

- Install selenium

Import the packages

```
import time
'''allow controlled delays'''
import csv
'''write tabular data to a CSV file'''
from pathlib import Path
'''safely handle file system paths'''
```

```
from selenium import webdriver
'''control the browser'''
from selenium.webdriver.chrome.options import Options
'''configure Chrome behavior'''
from selenium.webdriver.common.by import By
'''locate elements on a page'''
from selenium.webdriver.support.ui import WebDriverWait
'''to wait for conditions'''
from selenium.webdriver.support import expected_conditions as EC
'''to define wait logic'''
```

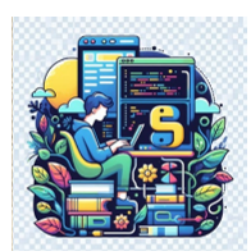


PROGRAMMING WITH PYTHON

Web Scraping – Dynamic Pages

- ```
URL = "https://airquality.cpcb.gov.in/AQI_India/"
'''stores the CPCB AQI dashboard address'''
OUT_CSV = "cpcb_repository_table.csv"
PROFILE_DIR = str(Path.cwd() / "chrome_profile_cpcb")
'''defines a persistent Chrome profile folder
A persistent profile allows cookies and captcha state to be reused'''
```

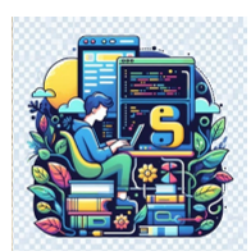




# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

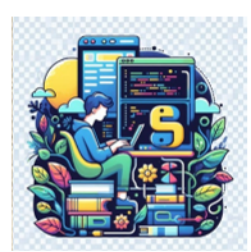
```
'''A Chrome Options object is created.'''
def start_driver():
 '''Chrome launch settings are stored inside this object'''
 options = Options()
 '''A Chrome argument is added and the browser viewport is forced to 1400 by 900 pixels.
 Page layouts become consistent across runs. Elements become easier to locate.'''
 options.add_argument("--window-size=1400,900")
 '''A Chrome argument is added for user data directory.
 PROFILE_DIR is inserted into the string using an f string.
 Chrome uses this folder to store cookies, cache, local storage, and session data.
 Captcha completion can be retained across runs when the same profile is reused.
 '''
 options.add_argument(f"--user-data-dir={PROFILE_DIR}")
 '''A Chrome argument is added to disable GPU acceleration.
 Fewer rendering issues are seen on some systems.
 Headless and virtualized runs become more stable.'''
 options.add_argument("--disable-gpu")
 '''A Chrome argument is added to reduce log output.
 ChromeDriver console noise is reduced.
 Only higher severity logs are shown'''
 options.add_argument("--log-level=3")
 '''A Chrome WebDriver is created using the Options object
 Chrome launches with all arguments you set above
 The driver object is returned to the caller for further automation'''
 return webdriver.Chrome(options=options)
```



# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

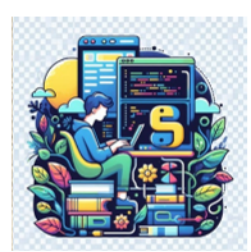
```
'''A function named extract_largest_table is defined.
driver is received as input.
driver is expected to be a Selenium WebDriver instance.'''
def extract_largest_table(driver):
 '''All table elements on the current page are collected.
 find_elements returns a list.
 By.TAG_NAME with value table matches every table tag'''
 tables = driver.find_elements(By.TAG_NAME, "table")
 '''hold the chosen table WebElement'''
 best = None
 '''stores the largest row count found so far'''
 best_rows = 0
 '''A loop is started over every table element found on the page'''
 for t in tables:
 '''For the current table t, all row elements are collected.
 tr is used because each table row is typically a tr tag.
 By.CSS_SELECTOR is used for flexible selection.'''
 rows = t.find_elements(By.CSS_SELECTOR, "tr")
 '''number of rows in the current table is compared with best_rows'''
 if len(rows) > best_rows:
 '''best_rows is updated to the current table's row count'''
 best_rows = len(rows)
 '''best is updated to the current table element'''
 best = t
```



# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

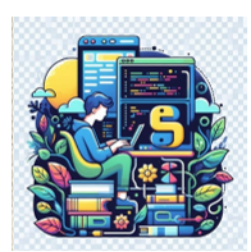
```
'''not best means no table was selected.
best_rows < 2 means only header or empty structure was found'''
if not best or best_rows < 2:
 '''stops execution because meaningful data is not present'''
 raise RuntimeError("No populated table was found.")
'''Rows are collected again from the selected table best'''
rows = best.find_elements(By.CSS_SELECTOR, "tr")
'''The header row is created. rows[0] targets the first row of the table.
th,td means header cells and normal cells are both accepted.
c.text extracts visible text from each cell.
strip removes extra spaces at the beginning and end.'''
header = [c.text.strip() for c in rows[0].find_elements(By.CSS_SELECTOR, "th,td")]
'''store all extracted data rows'''
data = []
'''A loop is started over rows after the header'''
for r in rows[1:]:
 '''All cells of the current row are collected.
 th,td is used to handle mixed header styled rows.
 text is extracted from each cell.
 strip removes extra spaces.
 replace converts line breaks inside a cell into a single space.
 This keeps each row as one clean CSV row'''
 cells = [c.text.strip().replace("\n", " ") for c in r.find_elements(By.CSS_SELECTOR, "th,td")]
 '''any(cells) is checked.
 True is returned if at least one cell has content'''
 if any(cells):
 '''The cleaned list of cells is appended to data'''
 data.append(cells)
'''header and data are returned as a pair
header is a list of column names.
data is a list of rows, where each row is a list of cell values'''
return header, data
```



# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

```
'''A function named close_blocking_modal is defined.
driver is received. It is your Selenium WebDriver.'''
def close_blocking_modal(driver):
 time.sleep(1)
 '''A list named close_xpaths is created.
 It stores multiple XPath patterns for possible close controls.'''
 '''An XPath is stored for a close button with class close inside the element with id myModal.
 This targets the common X icon.'''
 close_xpaths = [
 "//div[@id='myModal']//button[contains(@class,'close')]",
 "//div[@id='myModal']//button[contains(.,'Close')]",
 "//div[@id='myModal']//button[contains(.,'OK')]",
 "//div[@id='myModal']//button[contains(.,'I Agree')]",
 "//div[@id='myModal']//a[contains(.,'Close')]",
]
 '''Each XPath is assigned to xp'''
 for xp in close_xpaths:
 '''Click attempts are protected from failure.'''
 try:
 '''A WebDriverWait is created for up to 3 seconds.'''
 '''The condition requires the element located by the current XPath to be clickable.'''
 '''The locator tuple is built.
 By.XPATH is the strategy. xp is the XPath string.'''
 btn = WebDriverWait(driver, 3).until(
 EC.element_to_be_clickable((By.XPATH, xp))
)
```

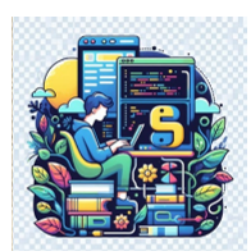


# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

```
'''JavaScript is executed in the page.
arguments[0] is replaced by btn.
click is triggered through JavaScript on that element.'''
driver.execute_script("arguments[0].click();", btn)
time.sleep(1)
return
except Exception:
 pass
'''The modal element with id myModal is searched
If the modal exists, it is removed from the DOM
The dark overlay behind the modal is searched
If the overlay exists, it is removed
The modal-open class is removed from the body
This restores page scrolling and click behavior '''
driver.execute_script("""
 const m = document.querySelector('#myModal');
 if (m) m.remove();
 const b = document.querySelector('.modal-backdrop');
 if (b) b.remove();
 document.body.classList.remove('modal-open');
""")
time.sleep(1)
```

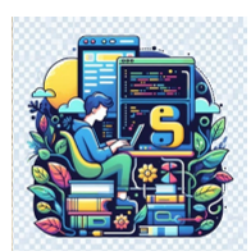




# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

```
'''driver is received as input.'''
def has_populated_table(driver):
 '''All table elements on the current page are collected using the tag name table.
 A loop is started. Each table element is assigned to t.'''
 for t in driver.find_elements(By.TAG_NAME, "table"):
 '''t.find_elements(By.CSS_SELECTOR, "tr") collects all row elements in the table.'''
 '''len(...) >= 2 checks that at least two rows exist.'''
 '''t.text.strip() checks that visible text exists in the table.'''
 if len(t.find_elements(By.CSS_SELECTOR, "tr")) >= 2 and t.text.strip():
 return True
 return False
```

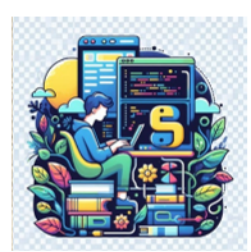


# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

```
'''A browser session is started'''
driver = start_driver()
'''All scraping logic is protected inside it'''
try:
 '''The target URL is loaded in the browser'''
 driver.get(URL)

 print("If captcha is shown, solve it. Then the repository link will be clicked.")
 '''An explicit wait is started.
 The browser is polled for page readiness.'''
 '''A lambda function is evaluated repeatedly.
 JavaScript is executed inside the page.'''
 '''document.readyState is checked.
 The value complete confirms full page load'''
 '''The wait block ends.
 Execution resumes only after the page loads.'''
 WebDriverWait(driver, 300).until(
 lambda d: d.execute_script("return document.readyState") == "complete"
)
```



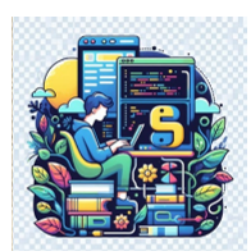
# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

```
'''An XPath locator is used.
Anchor tags containing specific text are targeted.'''
'''The link is required to be clickable.
This ensures it is visible and enabled.'''
'''The repository link element is returned.'''
repo_link = WebDriverWait(driver, 300).until(
 EC.element_to_be_clickable((By.XPATH, "//a[contains(., 'AQI Data Repository') or contains(., 'Repository')]"))
)
close_blocking_modal(driver)

driver.execute_script(
 "arguments[0].scrollIntoView({block:'center'});", repo_link
)
```

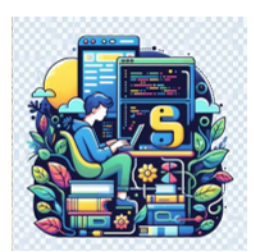




# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

```
'''A short sleep is applied'''
time.sleep(0.5)
'''JavaScript click is executed on repo_link.
 arguments[0] is replaced with repo_link.
 This bypasses Selenium's normal click, reducing interception issues.'''
driver.execute_script("arguments[0].click();", repo_link)
'''An explicit wait is started for up to 10 seconds
 The condition checks the number of open browser windows or tabs.'''
'''d.window_handles returns a list of window handles.
 len(...) >= 1 ensures at least one handle exists.
 This condition is almost always true, so it mainly acts as a short pause after the click.'''
WebDriverWait(driver, 10).until(lambda d: len(d.window_handles) >= 1)
'''A condition checks if more than one window handle exists.
 More than one means a new tab was opened.'''
if len(driver.window_handles) > 1:
 '''The browser context is switched to the last opened tab.
 driver.window_handles[-1] selects the newest tab.
 This ensures extraction runs on the repository page.'''
 driver.switch_to.window(driver.window_handles[-1])
```



# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages

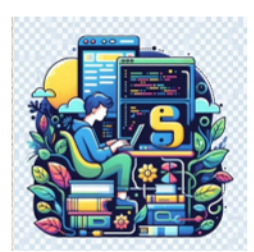
```
print("Repository page loaded")
print("URL:", driver.current_url)
print("Title:", driver.title)
'''A wait is started for up to 180 seconds.
 poll_frequency=1 means the check runs once per second.
 has_populated_table is called repeatedly until it returns True.'''
WebDriverWait(driver, 180, poll_frequency=1).until(has_populated_table)
'''extract_largest_table(driver) is called.
 The largest table is selected.
 Header and row data are returned.'''
header, rows = extract_largest_table(driver)

with open(OUT_CSV, "w", newline="", encoding="utf-8") as f:
 writer = csv.writer(f)
 writer.writerow(header)
 writer.writerows(rows)

print("Saved", len(rows), "rows to", OUT_CSV)

except Exception as e:
 Path("debug_repo_page.html").write_text(driver.page_source, encoding="utf-8")
 driver.save_screenshot("debug_repo_page.png")
 print("Extraction failed. Debug files saved.")
 raise

finally:
 '''closes all browser windows and ends the ChromeDriver session.'''
 driver.quit()
```



# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages - Output

National Air Quality Index

airquality.cpcb.gov.in/AQI\_India/

Chrome is being controlled by automated test software.

National Air Quality Index

Central Pollution Control Board,  
Ministry of Environment, Forests and Climate Change

State  City

Captcha Verification

IEDOZB

IEDOZB

Verify

[Click here for AQI Data Repository](#)

| AQI | Remark | Color Code | Possible Health Impacts |
|-----|--------|------------|-------------------------|
|-----|--------|------------|-------------------------|

[List of AQI Stations with Data of above selected Date & Time](#)

समर्पित। || Celebrating Golden Jubilee Year of CPCB (1974-2024)- 5 decades in pursuit of clean environment.



# PROGRAMMING WITH PYTHON

## Web Scrapping – Dynamic Pages - Output

National Air Quality Index x CCR x +

airquality.cpcb.gov.in/ccr/#/caaqm-dashboard-all/caaqm-landing/aqi-repository

Chrome is being controlled by automated test software.

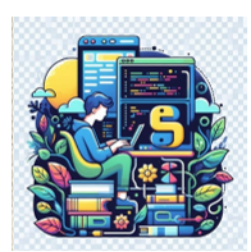
 Central Control Room for Air Quality Management - All India

AQI Repository

|           |           |      |            |
|-----------|-----------|------|------------|
| Frequency | Daily     | Type | City Level |
| State     | Karnataka | City | Bengaluru  |

Submit

ge the contribution of respective agencies in generating the data.



# PROGRAMMING WITH PYTHON

## Web Scraping – Dynamic Pages - Output

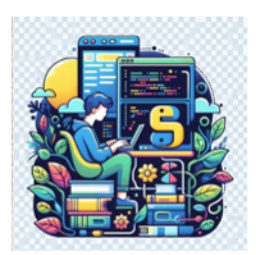
If captcha is shown, solve it. Then the repository link will be clicked.

Repository page loaded

URL: <https://airquality.cpcb.gov.in/ccr/#/caaqm-dashboard-all/caaqm-landing/aqi-repository>

Title: CCR

Saved 9 rows to cpcb\_repository\_table.csv



# PROGRAMMING WITH PYTHON

## Recap

---

- Web scraping
  - Static Pages
  - Dynamic Pages
- Output



**THANK YOU**

---

**Lekha A**  
Computer Applications